

CS 2110 Final Exam: LC-3 Assembly

Max Everest, Sebastian Jankowski, Jonathan Marto, Annelise Lloyd, Zaid Akkawi

Summer 2023

Contents

1	Rules - Please Read	2
2	Overview	2
2.1	Purpose	2
2.2	Task	2
2.3	Criteria	2
3	Instructions and Implementation	3
3.1	gcd algorithm	3
4	Grading	4
5	Deliverables	4
6	Local Autograder	5
7	LC-3 Assembly Programming Requirements and Tips	6
7.1	Overview	6
8	Appendix	7
8.1	Appendix A: LC-3 Instruction Set Architecture	7

Please take the time to read the entire document before starting the assignment. It is your responsibility to follow the instructions and rules.

1 Rules - Please Read

You are allowed to submit this portion of the final exam starting from the moment your timed lab/coding portion of the exam period begins until your individual period ends. You have 75 minutes to complete *both of the timed lab (LC3-Assembly and C)* portions of the exam, unless you have accommodations that have already been discussed with your professor. Gradescope submissions will close precisely at the end of your allotted exam period. **You are responsible for watching your own time.**

If you have questions during the exam, you may ask the TAs for clarification, though you are ultimately responsible for what you submit. The information provided in this document takes precedence. If you notice any conflicting information, please indicate it to your TAs.

The timed lab/coding section of the final exam is open-resource. You may reference your previous homeworks, class notes, etc., but your work must be your own. Contact in any form with any other person besides a TA is absolutely forbidden. **No collaboration is allowed.**

2 Overview

2.1 Purpose

The purpose of this coding section of the final exam is to test your understanding of coding in LC-3 assembly.

2.2 Task

In this section of the final exam, you will be implementing a short assembly program. Please see the detailed instructions on the following page. We have provided pseudocode for the program—you should follow the algorithm when writing your assembly code.

2.3 Criteria

This section will be graded based on your ability to correctly translate the given pseudocode for an algorithm into LC-3 assembly code. Please use the LC-3 instruction set when writing these programs. Check the Deliverables section for what you must submit to Gradescope.

You must produce and save the correct value for the algorithm, and your code must assemble with no warnings or errors (Complx and the autograder will tell you if there are any). If your code does not assemble, we will not be able to grade that file and you will not receive any points.

3 Instructions and Implementation

For the LC-3 Assembly coding section of the final exam, you will be implementing a subroutine that calculates the greatest common divisor (GCD) of two integers. The result should be stored on the stack in accordance with the LC-3 Calling Convention. You must implement the recursive version to get full credit. We provide a helper subroutine called MODULUS. It is encoded in hexadecimal, but you still can call it like any normal subroutine!

3.1 gcd algorithm

Implement the recursive Euclidean algorithm:

If b is 0, then the GCD is a . Else, try again with $a \leftarrow b$ and $b \leftarrow a \% b$.

We provide a helper subroutine MODULUS that can calculate $a \% b$ for you.

Pseudocode:

```
int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }
    c = modulus(a, b); // MODULUS subroutine has already been provided
    return gcd(b, c);
}
```

Example:

$\text{GCD}(56, 36)$

$36 \neq 0$

$\text{GCD}(36, 56 \% 36) = \text{GCD}(36, 20)$

$20 \neq 0$

$\text{GCD}(20, 36 \% 20) = \text{GCD}(20, 16)$

$16 \neq 0$

$\text{GCD}(16, 20 \% 16) = \text{GCD}(16, 4)$

$4 \neq 0$

$\text{GCD}(16, 16 \% 4) = \text{GCD}(4, 0)$

$0 = 0 \rightarrow \text{return } 4$

$\text{GCD}(56, 36) = 4$

Please refer to Grading for details on how gcd will be graded.

4 Grading

Point distribution for this coding portion of the final exam is broken down as follows:

- gcd (85 points): Correct return value.
- Calling Convention (15 points): Appropriately calls MODULUS and correctly follows LC-3 Calling Convention.

5 Deliverables

Turn in the following file on Gradescope during your assigned final exam coding period:

1. gcd.asm

6 Local Autograder

To run the autograder locally, follow the steps below depending upon your operating system:

- Mac/Linux Users:
 1. Navigate to the directory your files are in (**in your terminal on your host machine, not in the Docker container via your browser**)
 2. Run the command `sudo chmod +x grade.sh`
 3. Now run `./grade.sh`
- Windows Users:
 1. In Git Bash (or Docker Quickstart Terminal for legacy Docker installations), navigate to the directory your files are in
 2. Run `chmod +x grade.sh`
 3. Run `./grade.sh`

7 LC-3 Assembly Programming Requirements and Tips

7.1 Overview

1. Your code must assemble with **no warnings or errors**. To assemble your program, open the file with Complx. It will complain if there are any issues. **If your code does not assemble, you will get a zero for that file.**
2. Do not assume that your registers start out initialized to zero. Assume everything is initialized with random garbage data.
3. Do not add any comments beginning with @plugin or change any comments of this kind.
4. You should not use a compiler that outputs LC3 to do this assignment.
5. **Test your assembly.** Don't just assume it works and turn it in.
6. **Comment your code!** (not a hard requirement, but will make your life much easier) This is especially important in assembly, because it's much harder to interpret what is happening later, and you'll be glad you left yourself notes on what certain instructions are contributing to the code. Comment things like what registers are being used for and what less intuitive lines of code are actually doing. To comment code in LC-3 assembly just type a semicolon (;), and the rest of that line will be a comment.

Avoid stating the obvious in your comments, it doesn't help in understanding what the code is doing.

Good Comment

```
ADD R3, R3, -1      ; counter--
BRp LOOP            ; if counter == 0 don't loop again
```

Bad Comment

```
ADD R3, R3, -1      ; Decrement R3
BRp LOOP            ; Branch to LOOP if positive
```

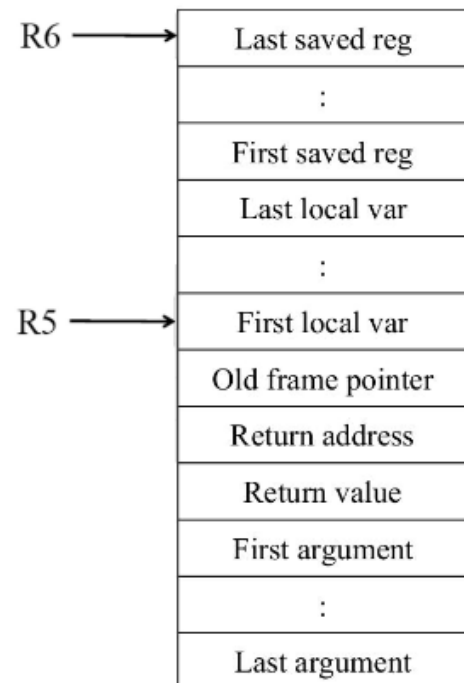
8 Appendix

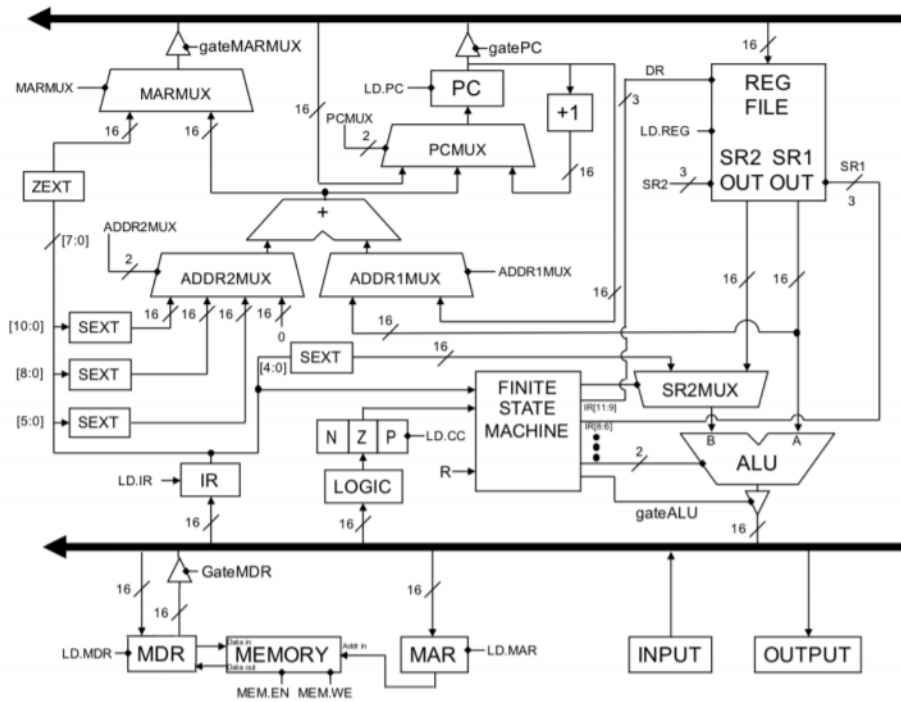
8.1 Appendix A: LC-3 Instruction Set Architecture

ADD	0001	DR	SR1	0	00	SR2
ADD	0001	DR	SR1	1	imm5	
AND	0101	DR	SR1	0	00	SR2
AND	0101	DR	SR1	1	imm5	
BR	0000	n	z	p	PCOffset9	
JMP	1100	000	BaseR	000000		
JSR	0100	1	PCOffset11			
JSRR	0100	0	00	BaseR	000000	
LD	0010	DR	PCOffset9			
LDI	1010	DR	PCOffset9			
LDR	0110	DR	BaseR	offset6		
LEA	1110	DR	PCOffset9			
NOT	1001	DR	SR	111111		
ST	0011	SR	PCOffset9			
STI	1011	SR	PCOffset9			
STR	0111	SR	BaseR	offset6		
TRAP	1111	0000		trapvect8		

Trap Vector	Assembler Name
x20	GETC
x21	OUT
x22	PUTS
x23	IN
x25	HALT

Device Register	Address
Keybd Status Reg	xFE00
Keybd Data Reg	xFE02
Display Status Reg	xFE04
Display Data Reg	xFE06





Boolean Signals	
LD.MAR	GateMARMUX
LD.MDR	GateMDR
LD.REG	GatePC
LD.CC	GateALU
LD.PC	LD.IR
MEM.EN	MEM.WE

MUX Name	Possible Values
ALUK	ADD, AND, NOT, PASSA
ADDR1MUX	PC, BaseR
ADDR2MUX	ZERO, offset6, PCOffset9, PCOffset11
PCMUX	PC+1, ADDER, BUS
MARMUX	ZEXT, ADDER
SR2MUX	SR2, SEXT